

Smart SSH Terminal

Andres Felipe Gonzalez Garzon, Haider Andres Cañon Murcia

Bogotá, Colombia

andres-gonzalez16@upc.edu.co, Haider-canon@upc.edu.co

Resumen—A lo largo de la historia de internet, los sistemas operativos han jugado un papel fundamental. En particular, los sistemas basados en Unix, como Linux, constituyen la columna vertebral de la infraestructura que sostiene la red global. Su administración se realiza principalmente a través de la terminal de línea de comandos. Este paper propone una solución innovadora que integra un modelo de lenguaje grande (LLM) en un cliente SSH, facilitando la interacción y el aprendizaje en el entorno de la terminal.

Index Terms—SSH, Linux, LLM, Terminal Inteligente, cliente SSH, Administración de Sistemas, Asistente de IA, Unix/Linux

I. INTRODUCCIÓN

Desde los orígenes de la computación moderna, los sistemas operativos han sido el puente entre el ser humano y la máquina. Sin importar la época ni el fabricante, todos ellos han compartido un rasgo común: una interfaz de comandos textual que, si bien otorga un control preciso y profundo sobre el sistema, exige al usuario memorizar una sintaxis extensa, interpretar mensajes de error crípticos y construir mentalmente un modelo abstracto de cómo funciona el sistema [1]. Este escenario, que décadas atrás era exclusivo de especialistas, sigue siendo hoy el punto de entrada obligado para cualquier persona que aspire a comprender y gestionar sistemas operativos basados en Unix/Linux [2]. El resultado es documentado empíricamente: los mensajes de error de las herramientas de línea de comandos presentan una dificultad sustancial para los usuarios novatos [3]; esta dificultad desencadena ciclos de frustración [4] que, si se repiten sin retroalimentación efectiva, producen desenganche emocional progresivo de la tarea —patrón documentado en novatos de programación ante errores repetidos [5] y estructuralmente análogo al aprendizaje de entornos CLI, donde la misma ausencia de retroalimentación inmediata opera como factor de abandono.

El auge reciente de los Modelos de Lenguaje Grande (LLMs) representa un punto de inflexión en este panorama. Por primera vez, es posible interactuar con sistemas complejos empleando lenguaje natural, recibir explicaciones contextuales al instante y obtener asistencia en la resolución de errores sin abandonar el entorno de trabajo [6], [7]. Esta capacidad abre una oportunidad sin precedentes para transformar la forma en que los estudiantes y técnicos aprenden a operar un sistema operativo: no a través de la memorización pasiva, sino mediante la práctica guiada, contextual y progresiva [8]. Sin embargo, en la práctica actual, la asistencia de IA permanece desacoplada del entorno donde ocurre el aprendizaje: el estudiante interrumpe su sesión de terminal, consulta un chatbot en el navegador y regresa al contexto de trabajo habiendo perdido

tanto el flujo cognitivo como la trazabilidad del error. Para que el potencial pedagógico de los LLMs se materialice, la inteligencia artificial debe integrarse de forma nativa en el mismo espacio donde ocurre la práctica: la terminal remota.

Precisamente de esa necesidad nace *Smart SSH Terminal*: un entorno de trabajo integrado que fusiona en una sola interfaz el acceso remoto seguro vía SSH, la gestión de archivos por SFTP y un asistente de IA contextual que acompaña al usuario en tiempo real. La herramienta está dirigida tanto a estudiantes de ingeniería como a técnicos en formación que requieren un andamiaje inteligente para desarrollar competencias reales en el manejo de Linux en entornos de laboratorio remoto con recursos limitados. Estos requisitos de eficiencia de memoria, bajo consumo de CPU y seguridad en las comunicaciones motivaron la elección de Rust y el framework Tauri como base tecnológica [9].

Las soluciones existentes abordan estos problemas de forma parcial: los clientes SSH tradicionales carecen de componente pedagógico; las plataformas de aprendizaje interactivo de Linux no ofrecen acceso real a infraestructura remota; y los terminales modernos con IA como Warp presentan limitaciones funcionales sobre sesiones SSH remotas y no incorporan mecanismos de andamiaje progresivo [10]. Ninguna herramienta integra simultáneamente acceso remoto autenticado, gestión de archivos y asistencia de IA pedagógica contextual en un único entorno. La Sección II documenta este vacío en detalle.

Las contribuciones principales de este trabajo son: (1) el diseño e implementación de *Smart SSH Terminal*, un cliente SSH con asistente de IA integrado de forma nativa que transforma los errores de ejecución en retroalimentación pedagógica contextual; (2) una arquitectura eficiente en Rust y Tauri que satisface los requisitos de cómputo de servidores de laboratorio de bajo coste; y (3) una evaluación empírica del sistema sobre infraestructura real, con métricas de rendimiento (uso de CPU, memoria y latencia de respuesta).

El resto del artículo se organiza así: la Sección II revisa los trabajos relacionados; la Sección III describe la arquitectura del sistema y sus subsistemas; la Sección IV justifica las decisiones de implementación, en particular la adopción de Rust; la Sección V presenta la metodología de pruebas y los resultados en el laboratorio remoto; finalmente, las Secciones VI y VII abordan el trabajo futuro y las conclusiones.

II. DISEÑO ARQUITECTÓNICO

II-A. Arquitectura General: Sistemas y Subsistemas

Smart SSH Terminal adopta una arquitectura de tres capas bien diferenciadas que se comunican de forma asíncrona a través del bridge nativo provisto por Tauri. Esta separación garantiza que los módulos puedan evolucionar y probarse de manera independiente, a la vez que el canal de comunicación entre ellos permanece tipado y seguro.

Como se ilustra en la Figura 1, el sistema se divide en tres subsistemas principales: el **Subsistema de Presentación** (Frontend), el **Subsistema de Núcleo** (Backend en Rust) y el **Subsistema de Inteligencia Artificial**.

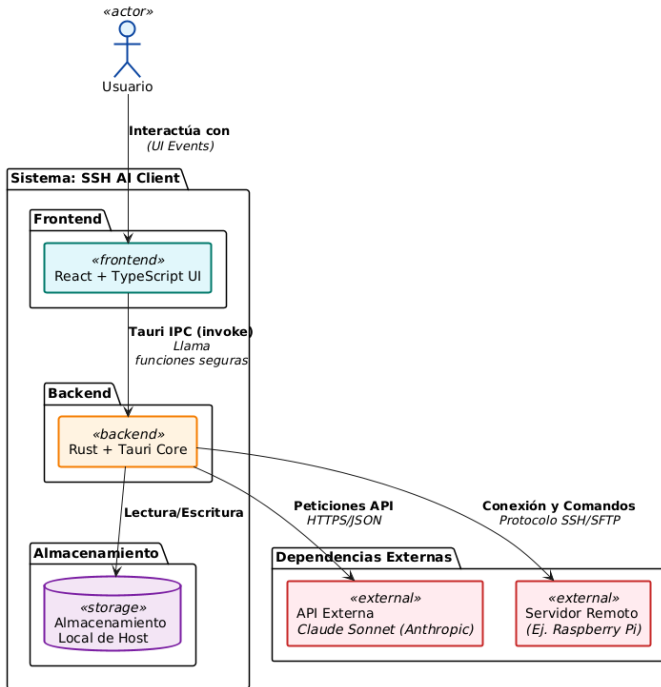


Figura 1. Arquitectura general del sistema, mostrando la interacción entre los tres subsistemas principales a través del bridge Tauri.

II-A1. Subsistema de Presentación (Frontend): Implementado en React con TypeScript, este subsistema es responsable de toda la interacción visual con el usuario. Se estructura en componentes reutilizables organizados por responsabilidad: componentes de terminal (TerminalView), gestión de archivos (FileManager), panel del asistente de IA (AiPanel), visualización de entorno gráfico remoto vía VNC (VncViewer) y panel de monitoreo de cámara. El estado de la aplicación se centraliza mediante React Context, eliminando el acoplamiento entre componentes y facilitando la actualización reactiva de la interfaz ante eventos SSH entrantes.

II-A2. Subsistema de Núcleo (Backend en Rust/Tauri): Es el motor del sistema y se implementa íntegramente en Rust, expuesto al frontend mediante comandos Tauri. Se subdivide en tres módulos fundamentales:

- **Módulo SSH** (`ssh/client.rs`): Gestiona el ciclo de vida de las conexiones SSH, incluyendo autenticación por contraseña y por clave pública, multiplexación de sesiones y manejo de canales de ejecución remota.
- **Módulo SFTP** (`cmd/sftp.rs`, `ssh/ssh2_sftp.rs`): Provee todas las operaciones de transferencia y gestión de archivos sobre el protocolo SFTP, incluyendo listado de directorios, carga, descarga, renombrado y eliminación.
- **Módulo de Trazabilidad** (`cmd/logs.rs`): Registra persistentemente cada comando ejecutado, respuesta del servidor, interacción con la IA y acción sobre archivos, conformando el historial de sesión utilizado en la fase de evaluación.

II-A3. Subsistema de Inteligencia Artificial: Implementado en los módulos `cmd/ai.rs`, `cmd/ai_utils.rs` y `cmd/agent.rs`, este subsistema actúa como intermediario entre el usuario y uno o más LLM externos (OpenAI GPT-4o, Google Gemini, o modelos locales vía Ollama). Recibe la consulta del usuario junto con el contexto de la sesión activa (comandos recientes, directorio actual, mensajes de error) y retorna una respuesta contextualizada que puede incluso proponerse como comando ejecutable directamente en la terminal.

II-B. Articulación de Servicios: Más Allá de la Terminal

Una limitación frecuente de los clientes SSH tradicionales es la necesidad de recurrir a herramientas externas para la gestión de archivos en servidores remotos, siendo WinSCP uno de los ejemplos más comunes en entornos educativos [?]. El diseño de *Smart SSH Terminal* elimina esta fragmentación al integrar de forma nativa tres categorías de servicio dentro de una sola interfaz unificada:

1. **Servicio de Terminal SSH:** Emulación completa de terminal con soporte de colores ANSI, pseudo-TTY y reenvío de señales al proceso remoto.
2. **Servicio de Gestión de Archivos SFTP:** Panel gráfico de exploración, carga y descarga de ficheros sobre el mismo canal SSH, sin necesidad de abrir una nueva conexión ni de utilizar una herramienta adicional.
3. **Servicio de Entorno Gráfico Remoto (VNC):** Integración de un visor noVNC embebido que permite acceder al escritorio gráfico del servidor remoto directamente desde la aplicación, útil en entornos de laboratorio con Raspberry Pi.

Esta tríada de servicios convierte a la herramienta en un entorno de trabajo completo, reduciendo el número de aplicaciones que el usuario debe gestionar simultáneamente y concentrando toda la actividad en un único punto de control trazable.

II-C. Diseño de la Interfaz y Trazabilidad

El diseño de la interfaz responde al principio de *supervisión progresiva*: la pantalla principal expone en primer plano la terminal y el asistente de IA, mientras que los paneles de gestión de archivos, cámara y entorno gráfico son accesibles

mediante pestañas colapsables, evitando la sobrecarga visual. La Figura 2 ilustra la distribución de paneles en la vista principal.

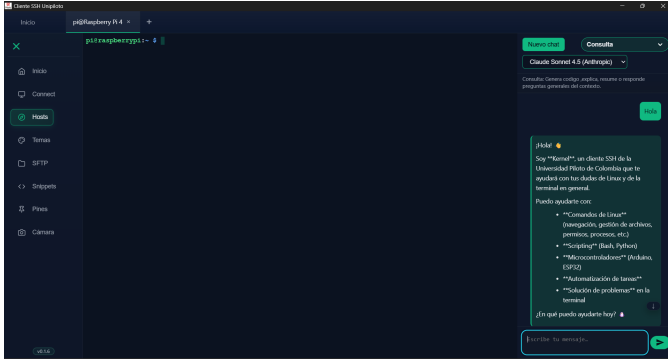


Figura 2. Vista principal de la interfaz: terminal SSH (izquierda), panel del asistente de IA (derecha) y barra de herramientas con acceso a servicios SFTP y VNC.

La trazabilidad se implementa a nivel de subsistema mediante el módulo `logs.rs`, que captura de forma estructurada cada evento relevante de la sesión: comando enviado, salida recibida, consulta a la IA, respuesta obtenida y acción SFTP realizada. Esta información se persiste localmente y sirvió como fuente primaria de datos cuantitativos durante las pruebas de usuario descritas en la Sección IV, permitiendo calcular métricas objetivas tales como la tasa de comandos autogenerados por IA, el tiempo entre consulta y ejecución, y el número de errores resueltos de forma autónoma.

III. IMPLEMENTACIÓN

III-A. Evolución del desarrollo

El desarrollo de esta herramienta comenzó inicialmente utilizando Tkinter, debido a que representaba la vía más rápida para prototipar una interfaz gráfica básica. Sin embargo, durante esta etapa se identificaron problemas críticos relacionados con el refresco de pantalla y la fluidez necesaria para un emulador de terminal en tiempo real.

III-B. Justificación de Rust

Ante las limitaciones de rendimiento encontradas, se decidió realizar un salto tecnológico hacia Rust [9]. Este lenguaje fue elegido por su enfoque en la seguridad de memoria [?], su alto rendimiento comparable al de C/C++ [?], y su ecosistema moderno que facilita la creación de aplicaciones de escritorio seguras. Rust permite una gestión eficiente de hilos y una respuesta inmediata en la interfaz de usuario.

III-C. Integración de servicios

La aplicación actual ya incorpora servicios equivalentes a los ofrecidos por herramientas tradicionales como WinSCP [?], permitiendo la gestión gráfica de archivos mediante el protocolo SFTP de forma integrada con la terminal.

III-D. La Novedad

La principal innovación radica en haber logrado una interfaz limpia y amigable que no solo emula una terminal, sino que es capaz de realizar un seguimiento inteligente a las acciones del usuario, ofreciendo asistencia contextual proactiva sin sobrecargar la experiencia de uso.

IV. ESCENARIO DE PRUEBAS

IV-A. Metodología

Para validar la herramienta, se llevó a cabo una metodología de pruebas con un grupo de 10 personas con distintos niveles de experiencia en Linux. El objetivo fue evaluar la facilidad de uso y la efectividad de la asistencia por IA.

IV-B. Métricas y gráficas

Los resultados mostraron tiempos de uso promedio de 20 minutos de interacción continua, durante los cuales los usuarios lograron generar comandos complejos y realizar tareas de administración que normalmente requerirían consultas externas a la documentación.

IV-C. Laboratorio remoto

La interfaz permite acoplarse a un escenario de laboratorio remoto [?], como el uso de una Raspberry Pi [?]. Esto permite a los usuarios desarrollar prácticas reales mientras monitorean el entorno físico. A continuación se presentan fotografías de evidencia del funcionamiento en este entorno.

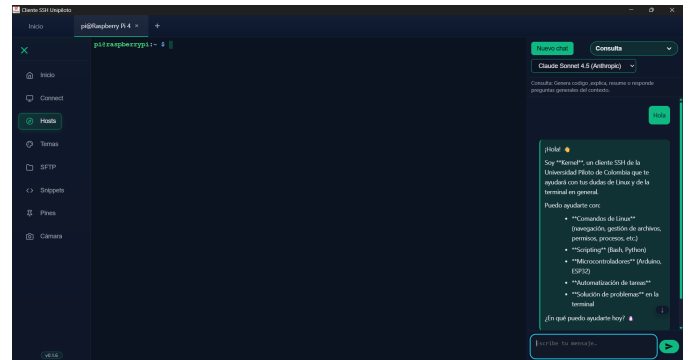


Figura 3. Interfaz del programa en ejecución.

V. TRABAJO FUTURO

Como líneas de trabajo futuro, se planea profundizar en la integración con modelos de IA, permitiendo al usuario elegir entre APIs comerciales (como ChatGPT de OpenAI o Gemini de Google) y soluciones de IA local (como Mistral) para garantizar la privacidad de los datos y evitar la sobrecarga del software con cálculos pesados en la nube.

Además, se propone añadir un botón de cambio rápido de idioma (español a inglés) directamente en la interfaz, facilitando su uso en entornos internacionales y bilingües.

AGRADECIMIENTOS

Se agradece a la Universidad, a los semilleros de investigación y a todos los integrantes del proyecto por el apoyo institucional y técnico brindado para la realización de este trabajo.

VI. CONCLUSIONES

El desarrollo de esta interfaz en Rust ha demostrado ser un éxito, proporcionando una herramienta robusta, segura y amigable para la administración de sistemas. El paper aporta una visión moderna de cómo la IA puede revitalizar el uso de la terminal.

Finalmente, las conclusiones se apoyan en la literatura existente que valida que "Linux y Rust van muy de la mano" [?], destacando la creciente adopción de Rust dentro del propio kernel de Linux y su relevancia en el desarrollo de herramientas de sistema de próxima generación.

REFERENCIAS

- [1] B. Shneiderman, "The future of interactive systems and the emergence of direct manipulation," *Behaviour & Information Technology*, vol. 1, no. 3, pp. 237–256, 1982.
- [2] D. M. Ritchie and K. Thompson, "The UNIX time-sharing system," *Communications of the ACM*, vol. 17, no. 7, pp. 365–375, 1974.
- [3] B. A. Becker, P. Denny, R. Pettit, D. Bouchard, D. J. Bouvier, B. Harrington, A. Kamil, A. Karkare, C. McDonald, P.-M. Osera, J. L. Pearce, and J. Prather, "Compiler error messages considered unhelpful: The landscape of text-based programming error message research," in *Proceedings of the Working Group Reports on Innovation and Technology in Computer Science Education (ITiCSE-WGR '19)*. ACM, 2019, pp. 177–210.
- [4] E. Ossovski and M. Brinkmeier, "Frustration tolerance among computer-science-related novice university students," *European Journal of Engineering Education*, vol. 49, no. 4, pp. 734–751, 2024.
- [5] N. Bosch and S. D'Mello, "The affective experience of novice computer programmers," *International Journal of Artificial Intelligence in Education*, vol. 27, no. 1, pp. 181–206, 2017.
- [6] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.
- [7] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [8] P. Denny, V. Kumar, and N. Giacaman, "Conversing with Copilot: Exploring prompt engineering for solving CS1 problems using natural language," in *Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE 2023)*. ACM, 2023, pp. 1136–1142.
- [9] S. Klabnik and C. Nichols, *The Rust Programming Language*. No Starch Press, 2019.
- [10] Warp Technologies, "Feature support over SSH," <https://docs.warp.dev/code/ssh-feature-support>, 2024, accessed: 2025.